

Tecnologia em Análise e Desenvolvimento de Sistemas
Desenvolvimento Web Back-end (BRADWBK)
Prof. Luiz Gustavo Diniz de Oliveira Véras

Orientações para o Desenvolvimento de Projetos Web Back-end

Introdução

Este documento descreve as especificações acerca do projeto a ser desenvolvido como requisito da disciplina de Desenvolvimento Web Back-end (BRADWBK). O foco é a construção de uma **API RESTful ou GraphQL** como servidor web, integrado a banco de dados relacional. Em adição, deve-se integrar no mesmo projeto um servidor baseado em eventos do tipo **WebSocket**.

Dinâmica do Trabalho

O sistema deverá ser desenvolvido por grupos de **1 até 6 alunos**.

Cada grupo deverá:

- Escolher um domínio (tema da aplicação);
- Especificar requisitos funcionais que possam envolver a API a ser desenvolvida. A API pode seguir o estilo RESTful ou GraphQL para serviços de requisição de dados. Especifique um requisito que possa incorporar o WebSocket no domínio especificado.
- Definir no mínimo **4 entidades do domínio** e envolvê-las em pelos menos um **relacionamento Um-para-muitos** e um **Muitos-para-Muitos**;
- Modelar e implementar a API, especificando as operações, tipos de corpo de requisição e de respostas, bem como os verbos a serem utilizados. Especifique de acordo com o estilo adotado para o projeto: RESTful ou GraphQL.
- A API modelada deve considerar operações de CRUD (CREATE-READ-UPDATE-DELETE) para cada uma das entidades.

- Realizar testes funcionais das rotas da API REST com ferramentas como **JMeter**, **Postman**, **Thunder Client, Insomnia**, entre outras.
- Para o caso da API WebSocket, crie páginas web para servirem de cliente na demonstração de seu uso.

Documento de Especificação da API

Cada grupo deverá elaborar um documento simples de especificação da API. O formato da especificação está separado por estilo de API. O Documento deve conter as seguintes informações.

- Domínio de aplicação da API (se for o mesmo de outra disciplina do curso, pode repeti-la aqui);
- Integrantes do grupo (nome e prontuário);
- Requisitos Funcionais da API
- Requisito Funcional para WebSocket
- Quatro entidades e a descrição do relacionamento entre elas;

A depender do estilo adotado no projeto da API, adicione mais uma informação no documento, como orientado nas seções a seguir.

RESTful

Para os projetos que adotarem a API Restful, deve-se criar uma tabela contendo as seguintes informações:

- Tabela de modelagem da API REST, conforme exemplo:

Verbo HTTP	Path	Body de Requisição (se houver)	Body de Retorno (se houver)	Status HTTP de sucesso	Status HTTP de erro

Obs: A Tabela anterior pode ser substituída por modelagem em ferramentas como Swagger, OpenAPI ou similar.

GraphQL

Para os projetos que adotarem a API GraphQL, deve-se criar uma tabela contendo as seguintes informações para mapear as operações disponíveis:

Tabela de modelagem da API GraphQL, conforme exemplo:

- Tipo de Operação (Query, Mutation ou Subscription)
- Nome do Campo (Nome da Query ou Mutation)
- Argumentos / Input (Variáveis e tipos de entrada, se houver)
- Tipo de Retorno / Payload (Tipo de objeto ou escalar retornado)
- Erros Possíveis (Mapeamento de exceções de negócio tratadas na propriedade errors)

Tipo de Operação	Nome do campo	Argumentos/ Inputs (se houver)	Tipo de retorno	Erros

Obs: A tabela anterior pode ser substituída pela especificação do Schema GraphQL documentada no formato SDL (Schema Definition Language) ou por ferramentas de documentação interativa como Apollo Studio, GraphiQL, Altair ou similar.

Tecnologias

As seguintes tecnologias são preferíveis, mas **podem ser substituídas**, desde que mantenham a aderência aos requisitos de projeto definidos aqui e a arquitetura REST:

- **Back-end:** Java com Spring Boot, Spring Web, Spring GraphQL, Spring WebSocket;
- **Banco de Dados**: PostgreSQL, MySQL ou outro banco relacional. Se preferir por utilizar o banco de dados H2, que funciona de forma similar ao SQLite, porém é para Java;
- **ORM**: JPA/Hibernate para operações com banco de dados relacional;
- **Teste Funcional**: JMeter, Postman, Thunder Client, Insomnia ou similar;
- **Versionamento de Código**: Git (preferencialmente GitHub, GitLab ou Bitbucket).

Entregas

O grupo deverá entregar:

- Documento de especificação do domínio, entidades e API Web em PDF;
- Código-fonte da API em repositório Git;
- Evidência dos testes funcionais com alguma ferramenta especializada. Podem ser geradas imagens JPEG ou PNG anexadas ao documento PDF do item anterior.

Data de Entrega de todos os documentos, códigos e apresentação em vídeo:
25 de novembro de 2026.

Algumas orientações adicionais para a apresentação:

- A documentação poderão ser agregadas em slides e entregue no dia;
- É necessário que o funcionamento do *back-end* seja demonstrado. Para tal, o grupo pode gravar vídeo para uso no dia da apresentação;
- O tempo máximo de vídeo para apresentação por grupo será de até **15 minutos**;
- Todos os membros do grupo devem falar no vídeo, caso contrário terão a nota obtida pelo grupo reduzida em **35%**. O aluno deve se identificar quando iniciar sua fala. Não é necessário que o estudante apareça no vídeo.

Critérios de Avaliação

Código do Critério	Descrição	Pontuação Máxima
CR1	Documento de especificação do domínio, entidades e da API Web	1,5
CR2	Operações de CRUD da entidade 1 pela APÌ Web integrada ao Banco de Dados com ORM	1,0
CR3	Operações de CRUD da entidade 2 pela APÌ Web integrada ao Banco de Dados com ORM	1,0
CR4	Operações de CRUD com a entidade 3 pela APÌ Web integrada ao Banco de Dados com ORM	1,0
CR5	Operações de CRUD com a entidade 4 pela APÌ Web integrada ao Banco de Dados com ORM	1,0
CR6	Implementação de 1 relacionamento Um-para-Muitos	1,0
CR7	Implementação de 1 relacionamento Muitos-para-Muitos	1,0
CR8	Implementação e Demonstração de funcionalidade com WebSocket	1,5
CR9*	Testes funcionais documentados com Thunder Client, Postman ou equivalente	1,0
Total da pontuação		**10,0**

Um ótimo desenvolvimento a todos!